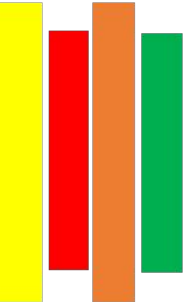


Outline

- ✓ Introduction
- ✓ Minimax
- ✓ Alpha Beta Pruning

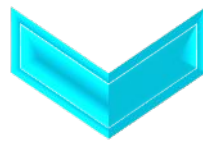
In which we examine the problems that arise when we try to plan ahead in a world where other agents are planning against us.

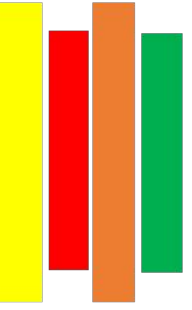


Introduction

□ Adversarial Search

- Multi-agent environment:
 - ✓ any given agent needs to consider the actions of other agents and how they affect its own welfare
 - ✓ Introduce possible contingencies into the agent's problem-solving process
- In this chapter we cover competitive environments, in which the agents' goals are in conflict, giving rise to adversarial search problems—often known as games.
- Mathematical game theory, a branch of economics, views any multiagent environment as a game, provided that the impact of each agent on the others is “significant,” regardless of whether the agents are **cooperative or competitive**.

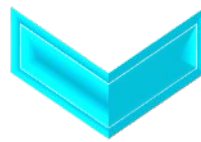


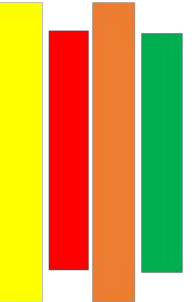


Types of Games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleship Kriegspiel	bridge, poker, scrabble nuclear war

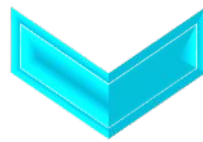
Not Considered: Physical games like tennis, croquet, ice hockey, etc.
(but see “robot soccer” <http://www.robocup.org/>)

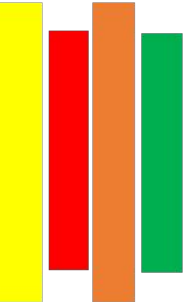




Assumptions

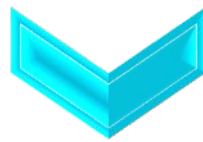
- The game state is **fully observable**.
- The outcome of each move is **deterministic**.
- Game states **discrete, finite** number of possible moves and game states.
- There are **no infinite runs** of the game: a terminal state is always reached after a finite number of steps.
- **Two-player zero-sum game**: two players, terminal states have utility with $\text{utility}(\text{player1}) = -\text{utility}(\text{player2})$.
- Our formulation (equivalent): single **utility function u** , players **Max** vs. **Min** trying to **maximize vs. minimize u** .
- **Turn-taking**: Players move alternately. Max begins.
- Compare to, e.g., “Prisoner’s Dilemma”
 - “Deterministic, Non-turn-taking, Non-Zero-sum game of Imperfect information”





Game vs Search Problems

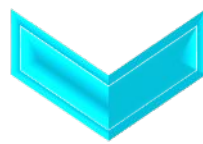
- Search – no adversary
 - Solution is (heuristic) method for finding goal
 - Heuristics and CSP techniques can find *optimal* solution
 - Evaluation function: estimate of cost from start to goal through given node
 - Examples: path planning, scheduling activities
- Games – adversary
 - Solution is strategy
 - strategy specifies move for every possible opponent reply.
 - Time limits force an *approximate* solution
 - Evaluation function: evaluate “goodness” of game position
 - Examples: chess, checkers, backgammon



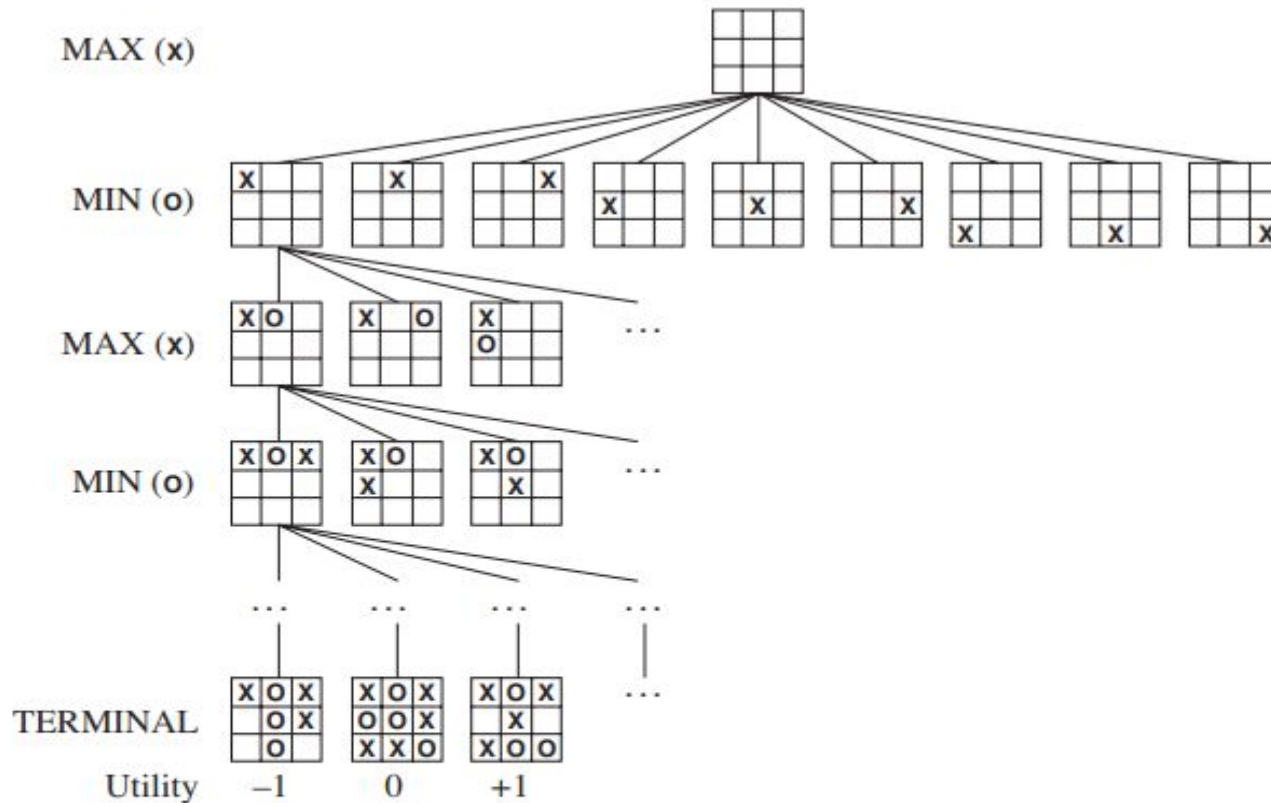


Game Problem Formulation

- S_0 : The **initial state**, which specifies how the game is set up at the start.
- $\text{PLAYER}(s)$: Defines which player has the move in a state.
- $\text{ACTIONS}(s)$: Returns the set of legal moves in a state.
- $\text{RESULT}(s, a)$: The **transition model**, which defines the result of a move.
- $\text{TERMINAL-TEST}(s)$: A **terminal test**, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**.
- $\text{UTILITY}(s, p)$: A **utility function** (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p . In chess, the outcome is a win, loss, or draw, with values $+1$, 0 , or $\frac{1}{2}$. Some games have a wider variety of possible outcomes; the payoffs in backgammon range from 0 to $+192$. A **zero-sum game** is (confusingly) defined as one where the total payoff to all players is the same for every instance of the game. Chess is zero-sum because every game has payoff of either $0 + 1$, $1 + 0$ or $\frac{1}{2} + \frac{1}{2}$. “Constant-sum” would have been a better term, but zero-sum is traditional and makes sense if you imagine each player is charged an entry fee of $\frac{1}{2}$.



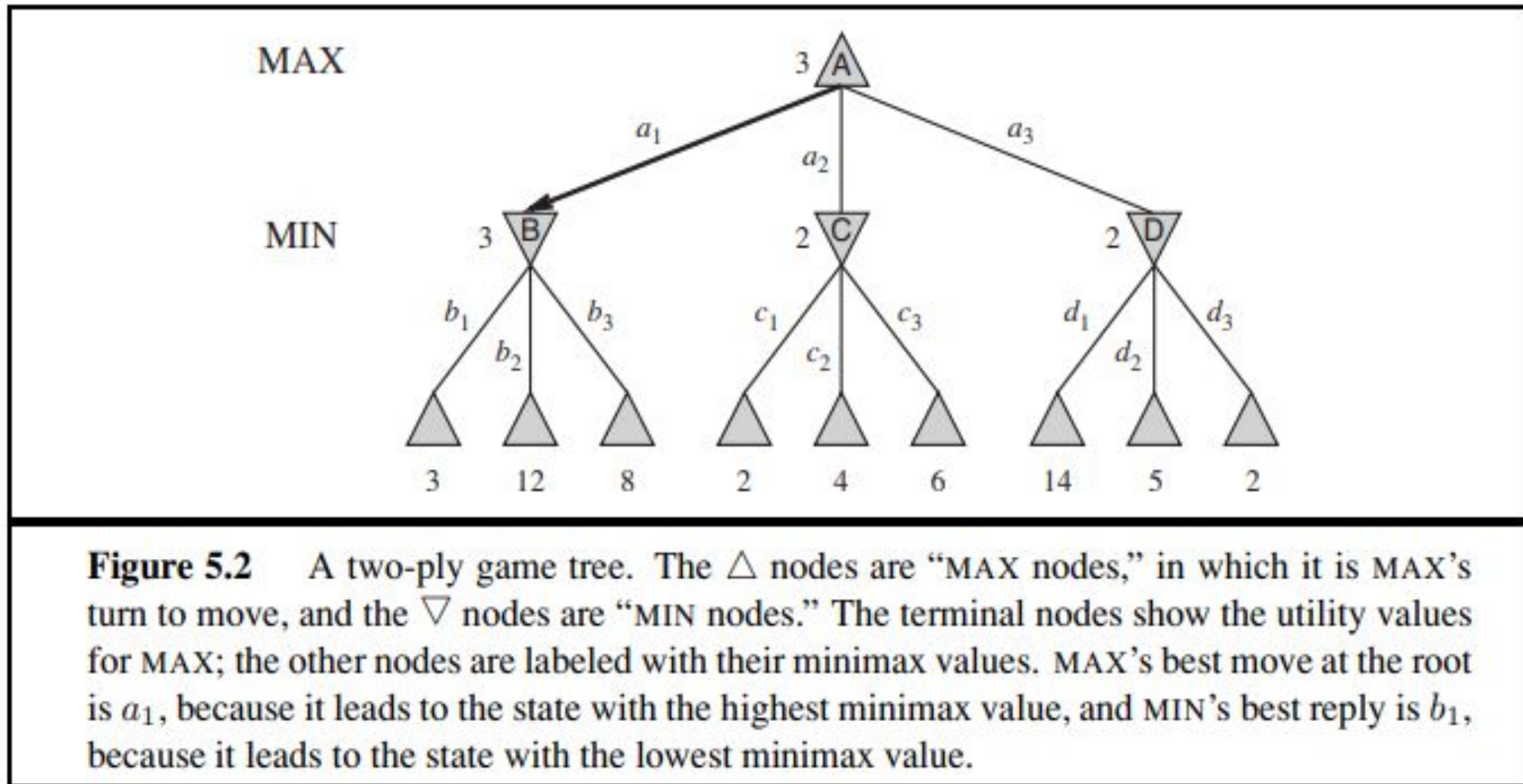
Game tree (2-player, deterministic, turns)



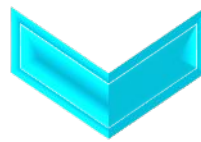
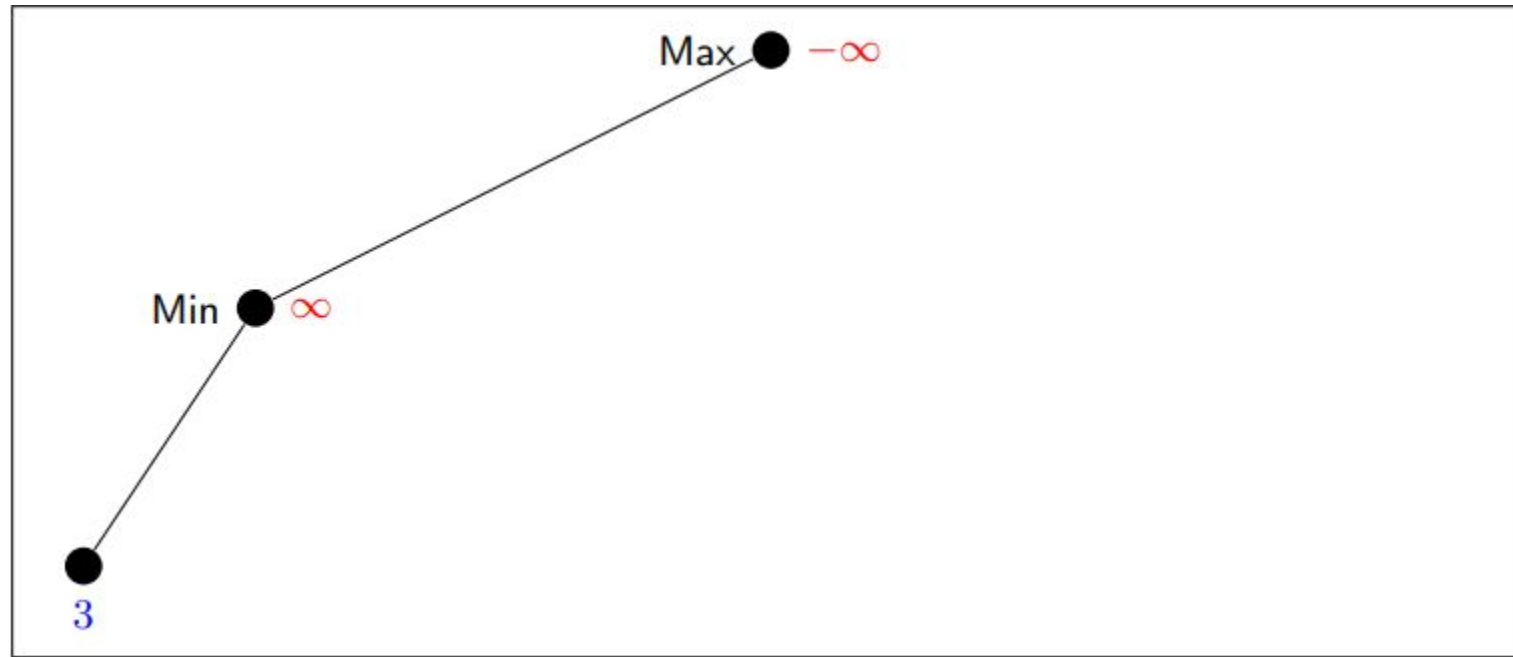
How do we search this tree to find the optimal move?

Figure 5.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

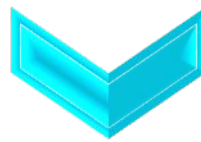
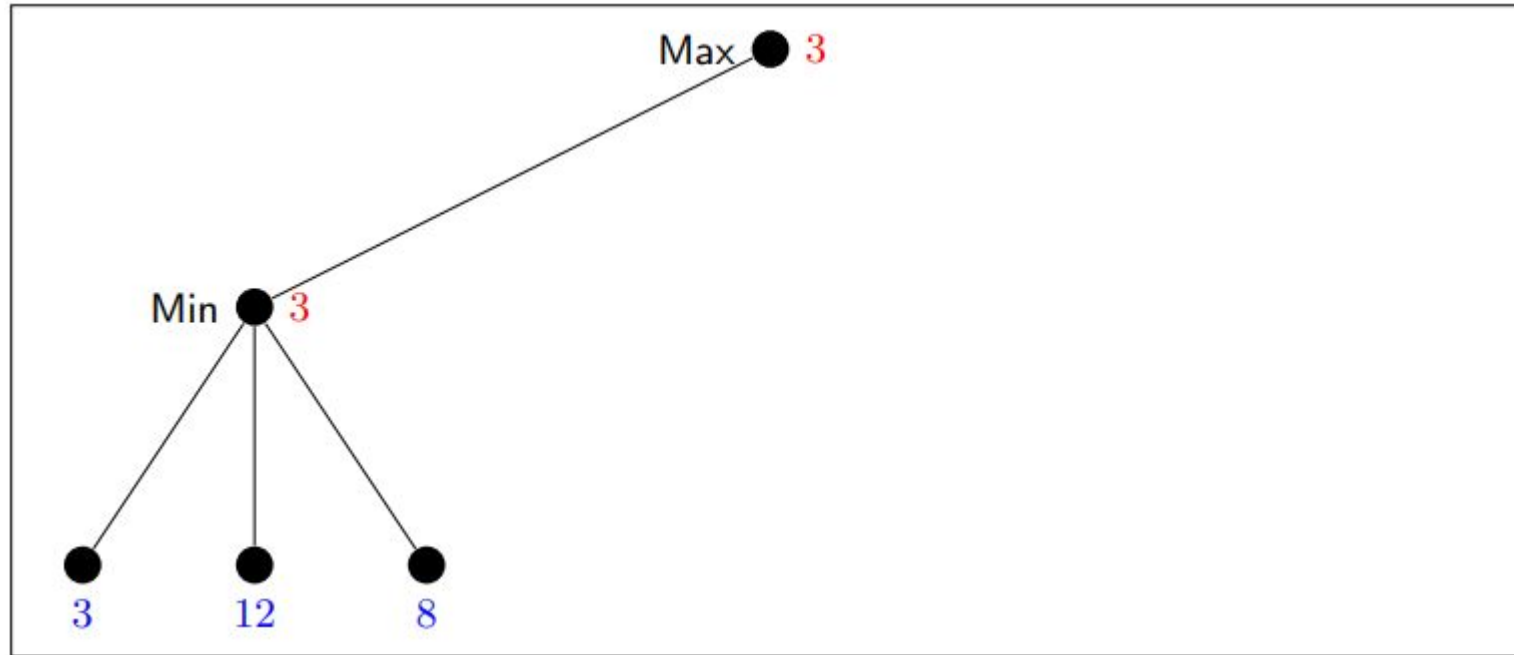
Minimax Strategy



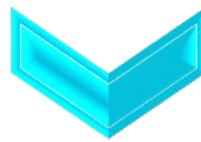
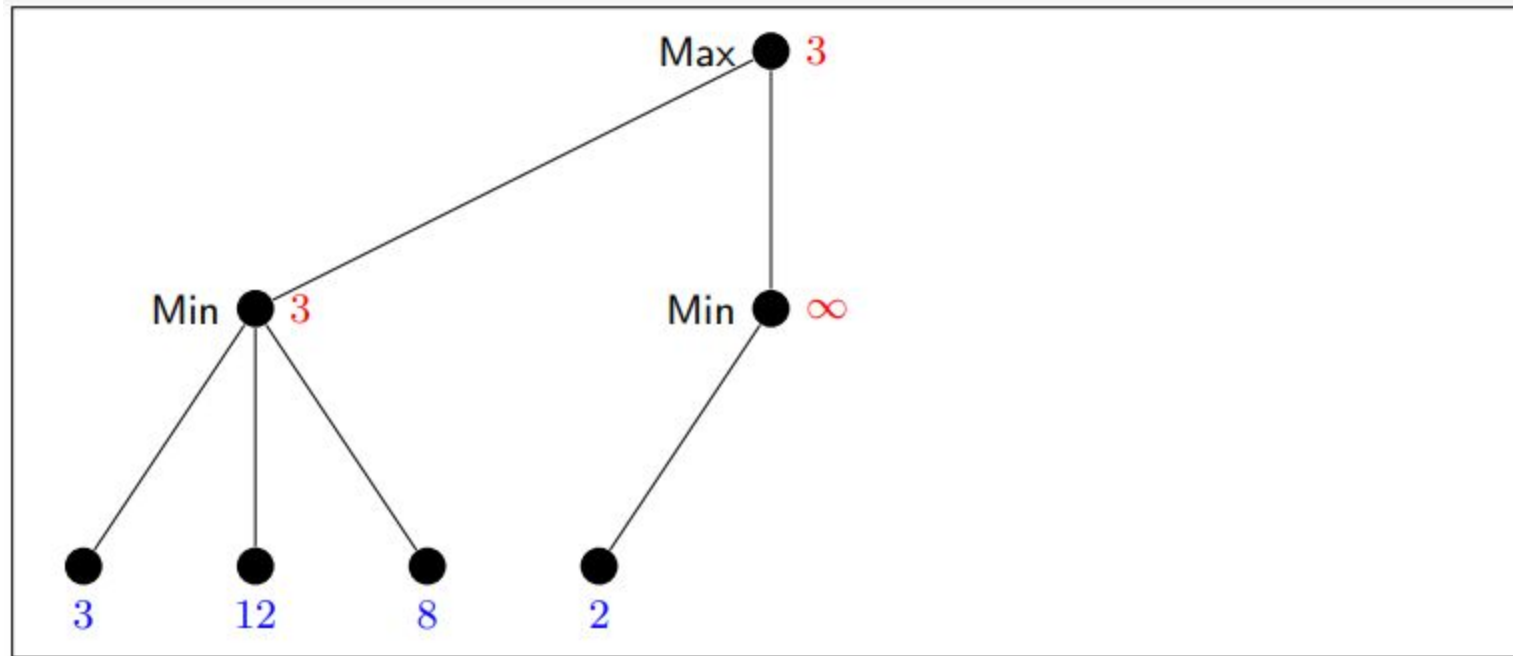
Minimax Details



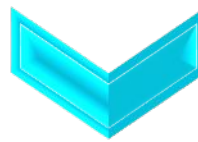
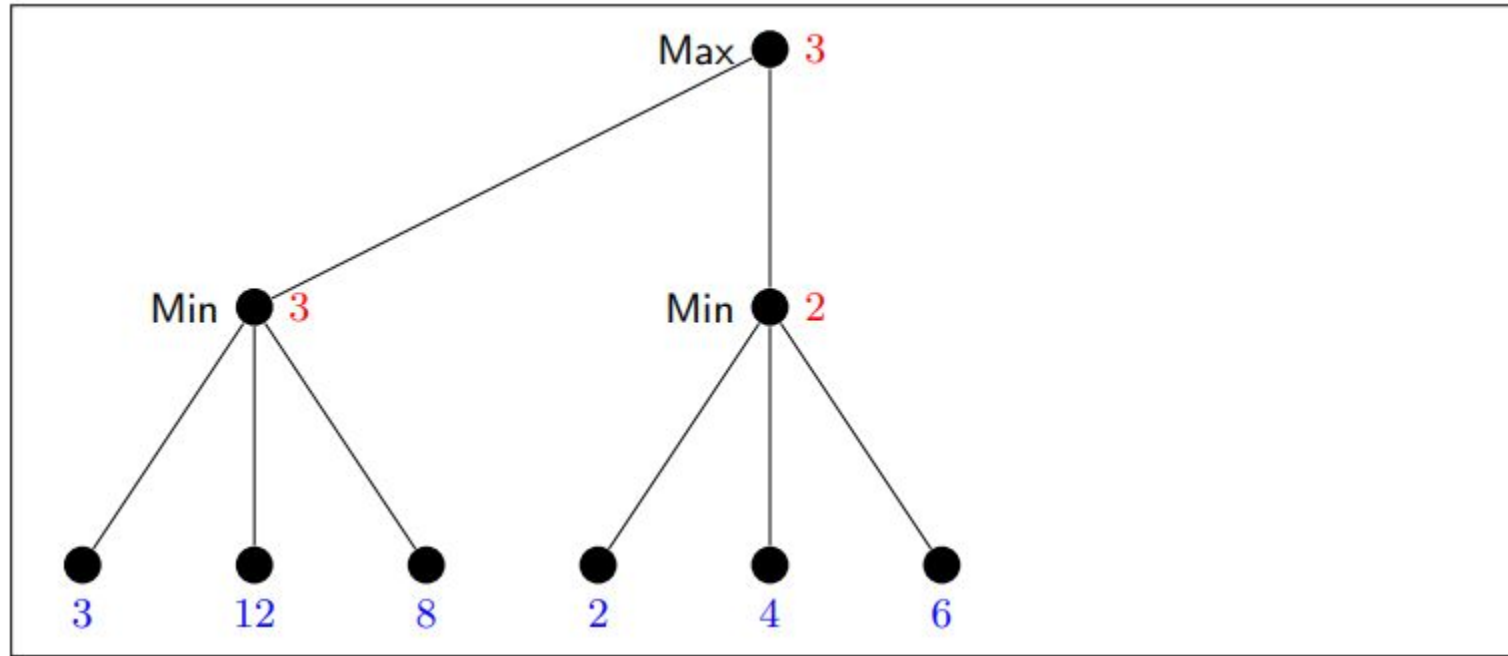
Minimax Details



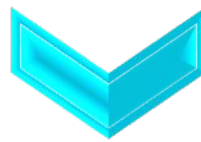
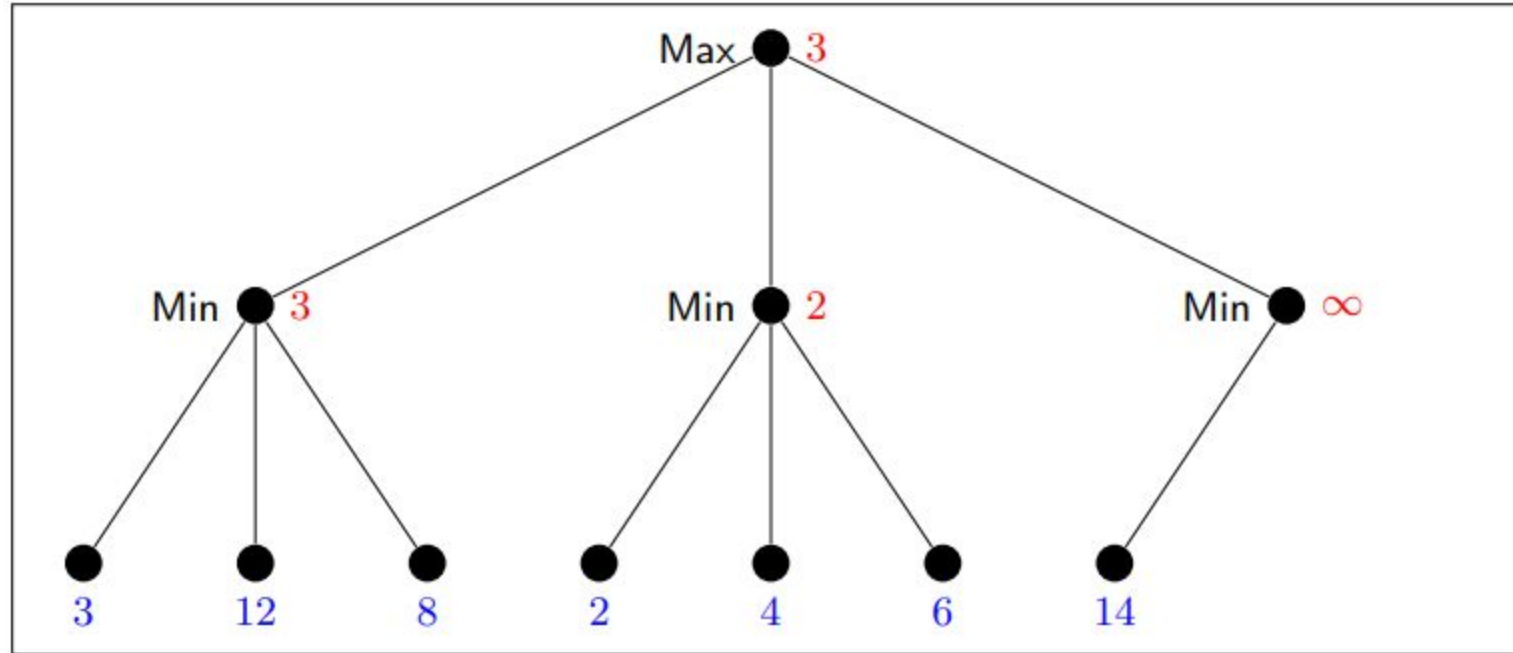
Minimax Details



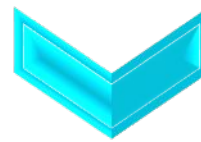
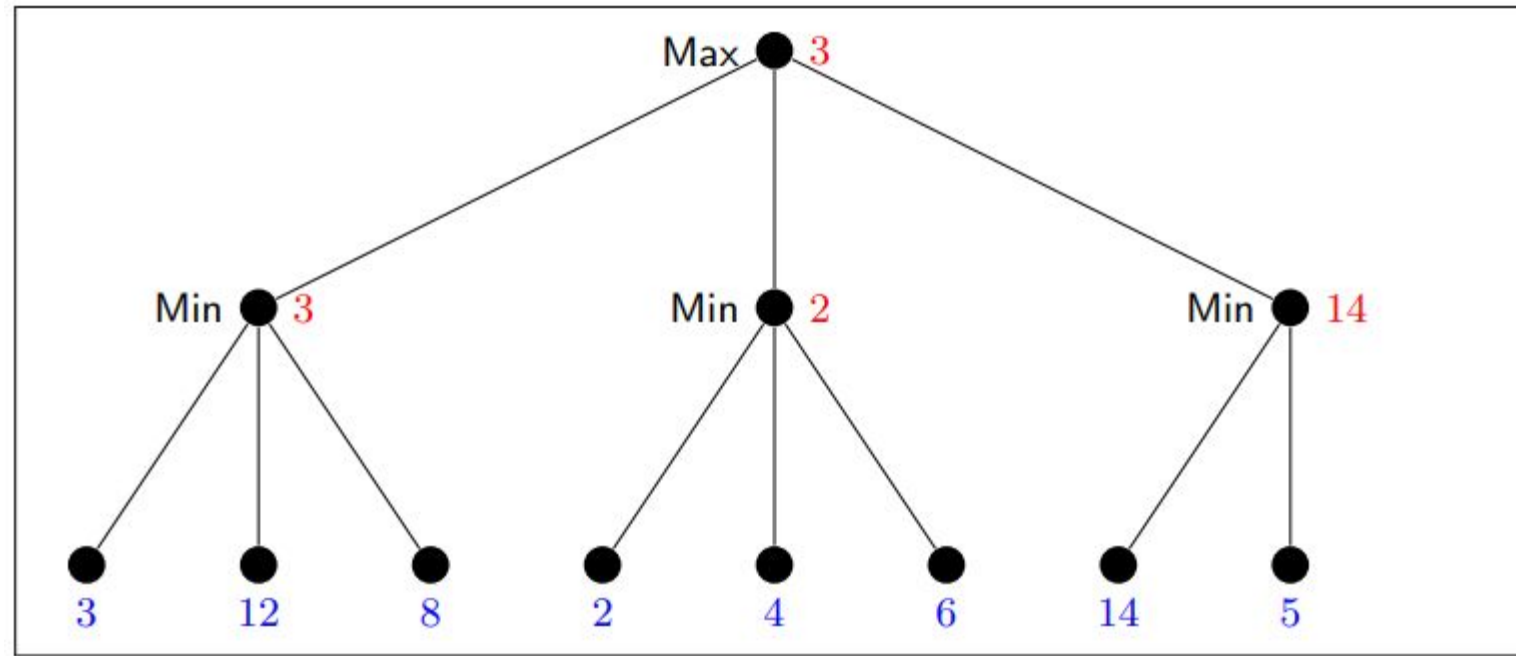
Minimax Details



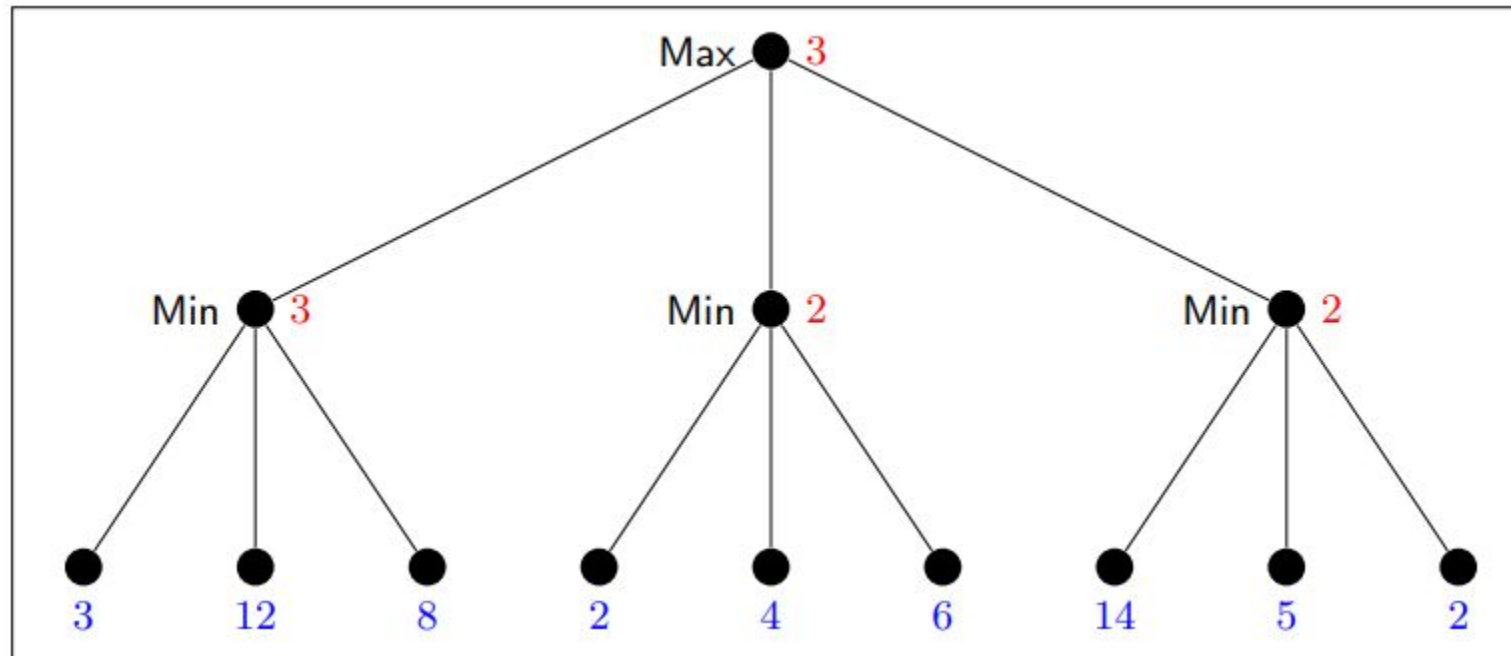
Minimax Details



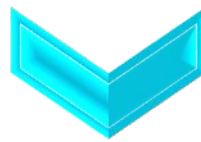
Minimax Details

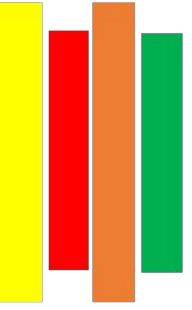


Minimax Details



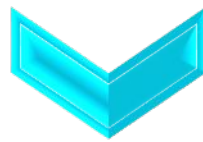
→ So which action for Max is returned? Leftmost branch. Note: The maximal possible pay-off is higher for the rightmost branch, but assuming perfect play of Min, it's better to go left. (Going right would be “relying on your opponent to do something stupid”.)





Analysis

- A complete depth-first search? **Yes**
- Time complexity? $O(b^d)$
- Space complexity? $O(bd)$ (depth-first exploration)
- For chess, $b \approx 35$, $d \approx 100$ for "reasonable" games
- Exact solution completely infeasible



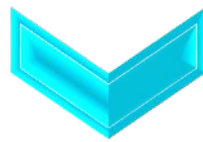


Alpha-Beta Pruning



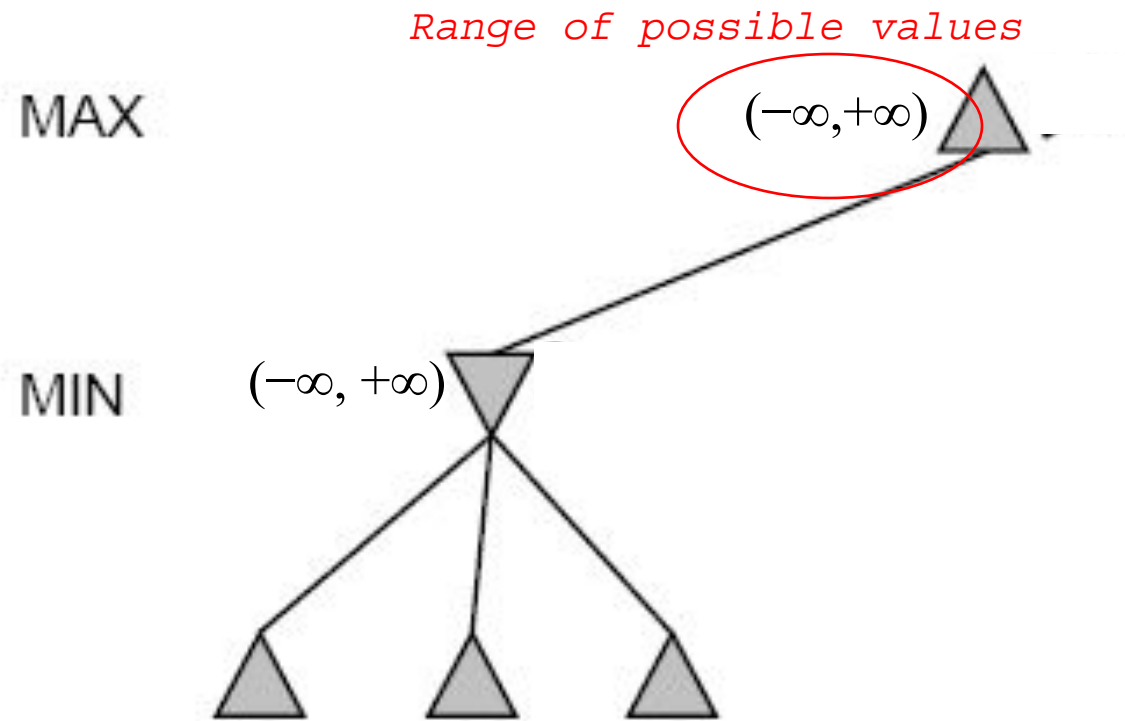
α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.

β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

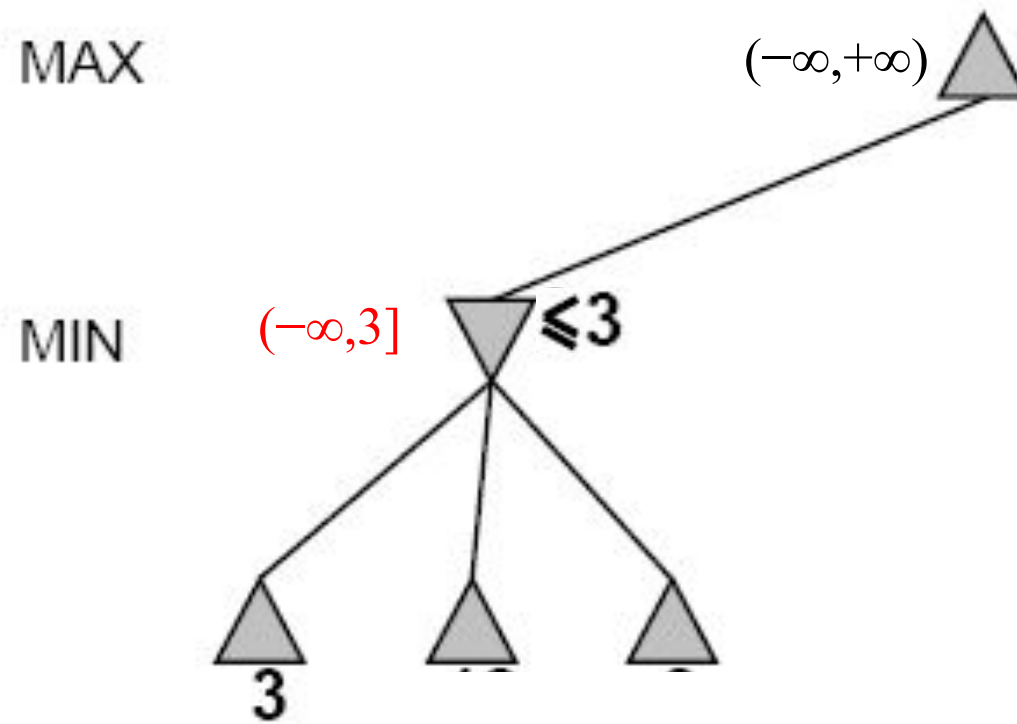


$\alpha - \beta$ Pruning Example

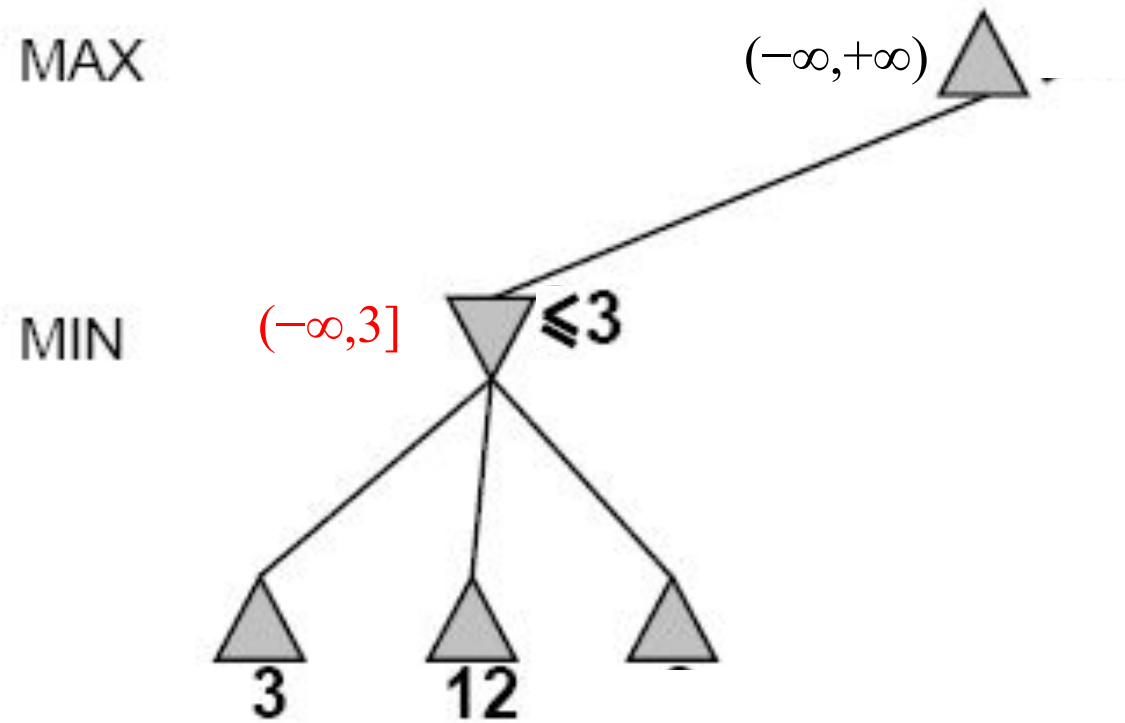
Do DFS until first leaf



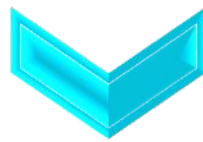
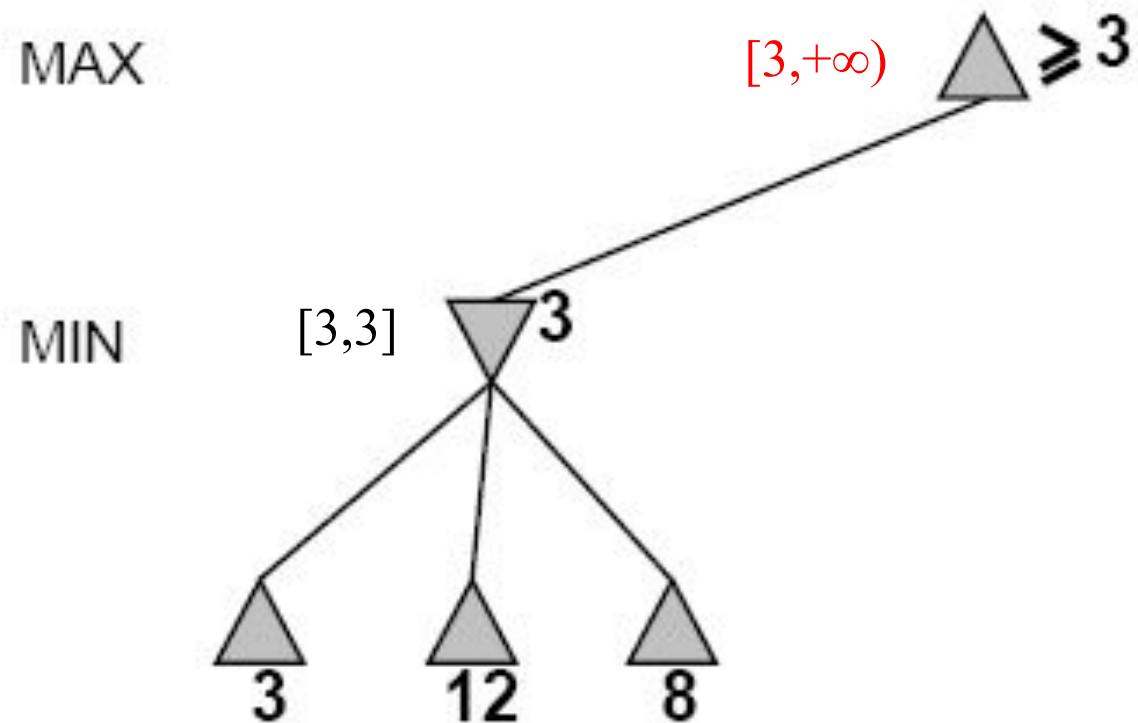
$\alpha - \beta$ Pruning Example



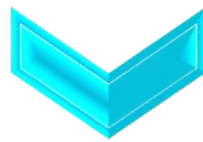
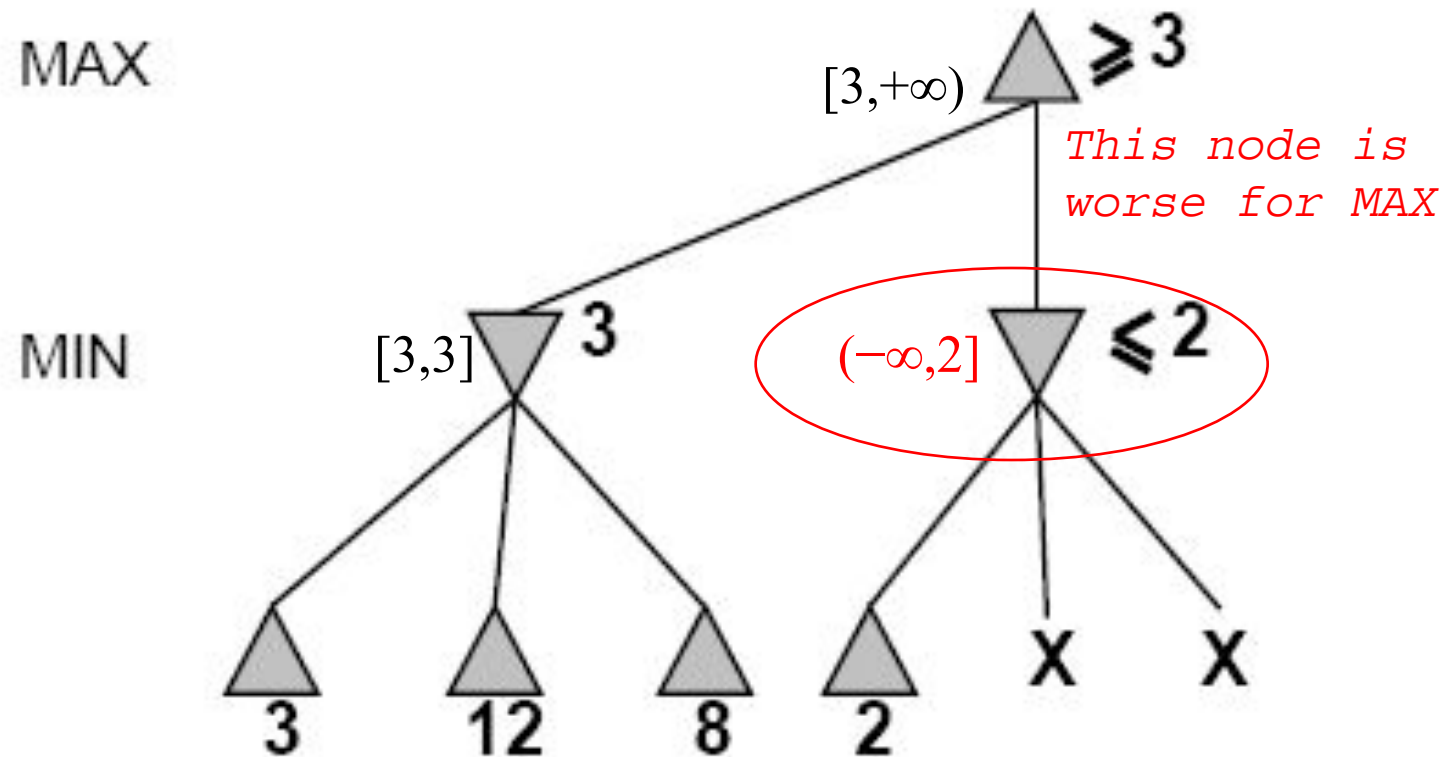
$\alpha - \beta$ Pruning Example



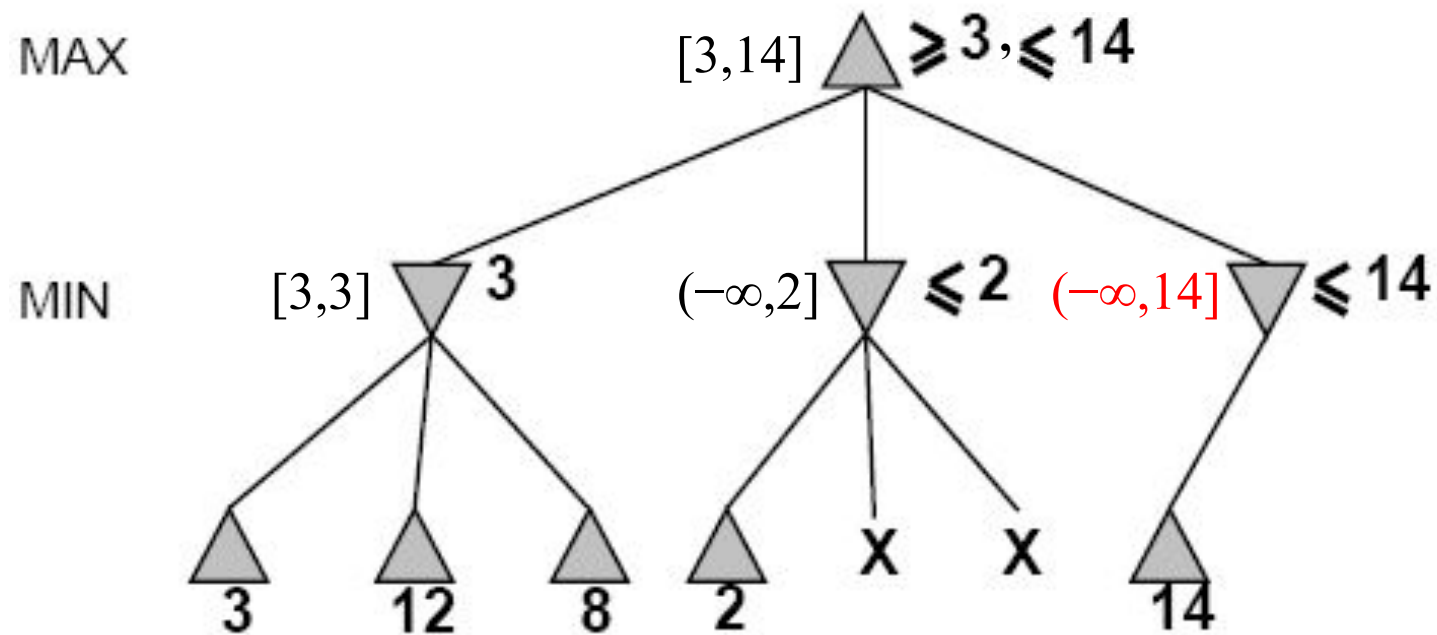
$\alpha - \beta$ Pruning Example



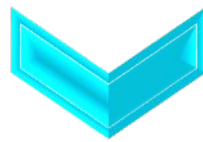
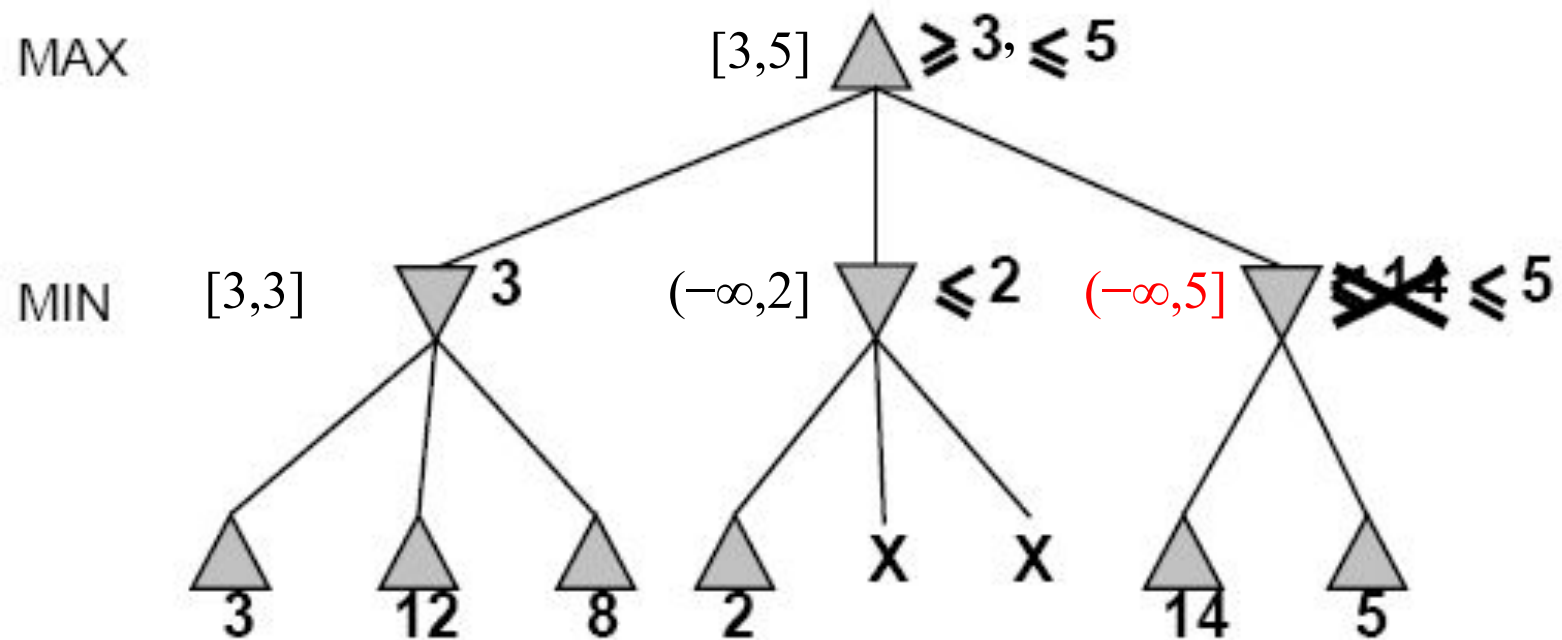
$\alpha - \beta$ Pruning Example



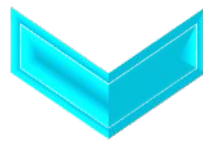
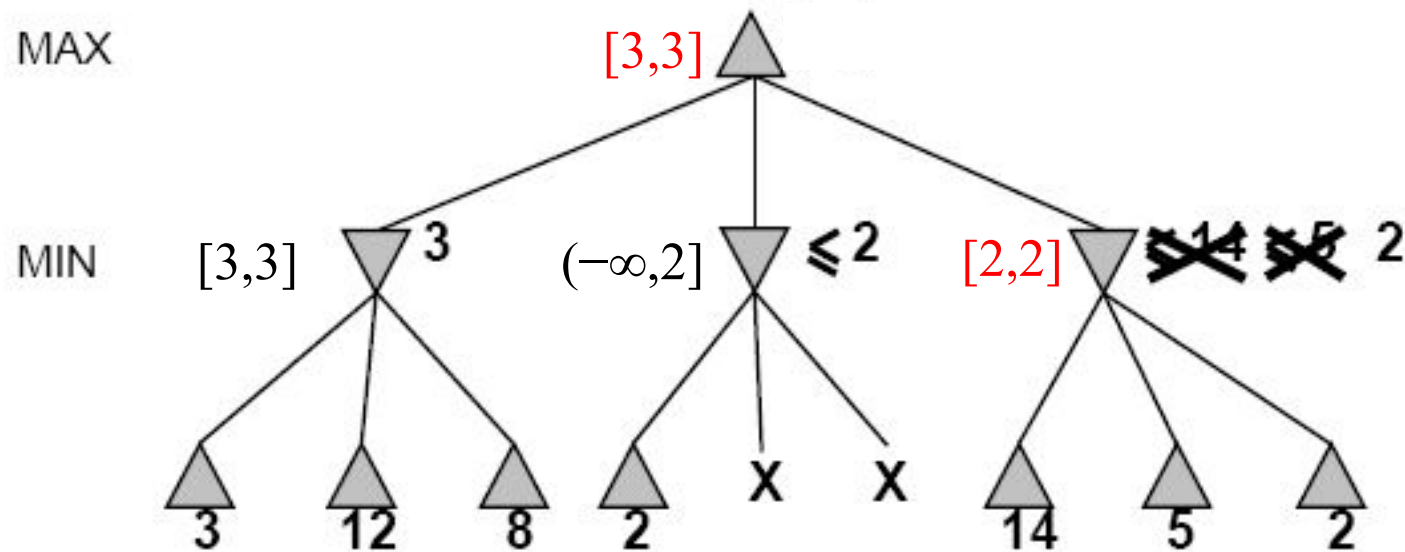
$\alpha - \beta$ Pruning Example



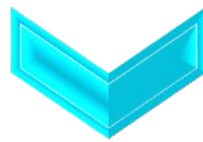
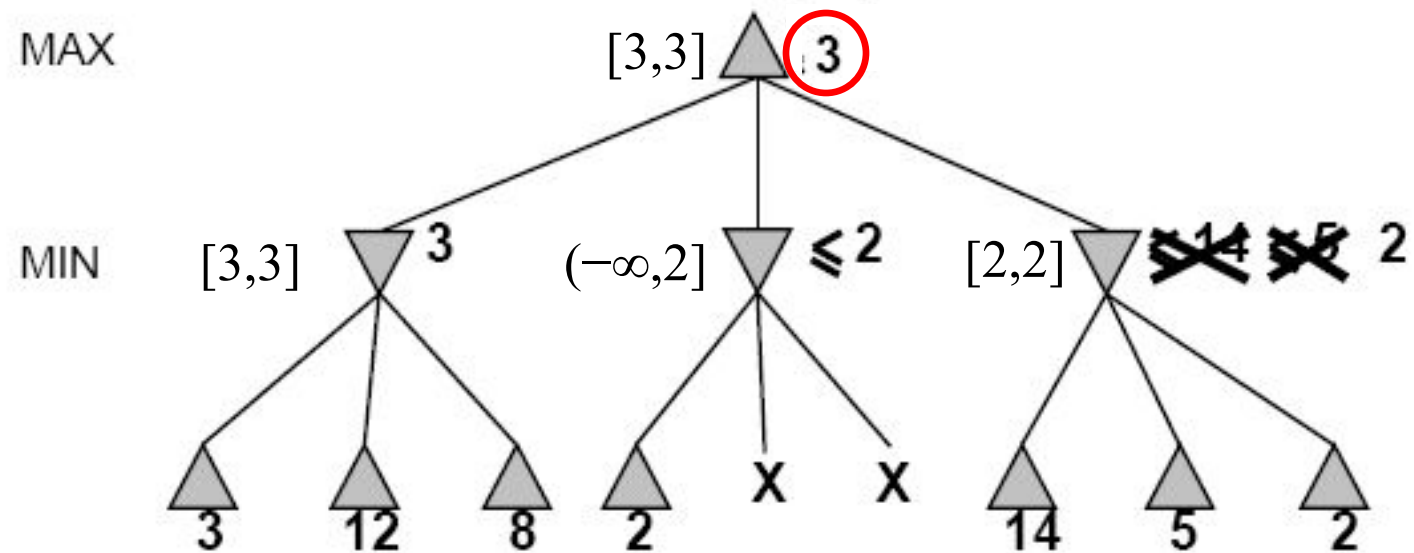
$\alpha - \beta$ Pruning Example

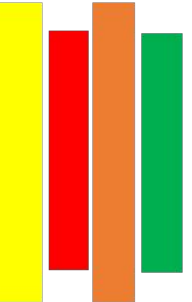


$\alpha - \beta$ Pruning Example



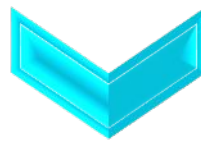
$\alpha - \beta$ Pruning Example

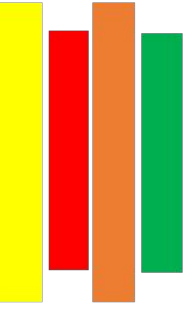




Alpha-beta Algorithm

- Depth first search
 - only considers nodes along a single path from root at any time
- α = highest-value choice found at any choice point of path for MAX
(initially, $\alpha = -\text{infinity}$)
- β = lowest-value choice found at any choice point of path for MIN
(initially, $\beta = +\text{infinity}$)
- Pass current values of α and β down to child nodes during search.
 - Update values of α and β during search:
 - MAX updates α at MAX nodes
 - MIN updates β at MIN nodes
 - Prune remaining branches at a node when $\alpha \geq \beta$

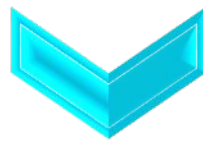




When to Prune

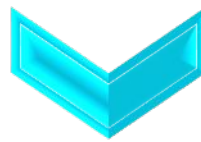
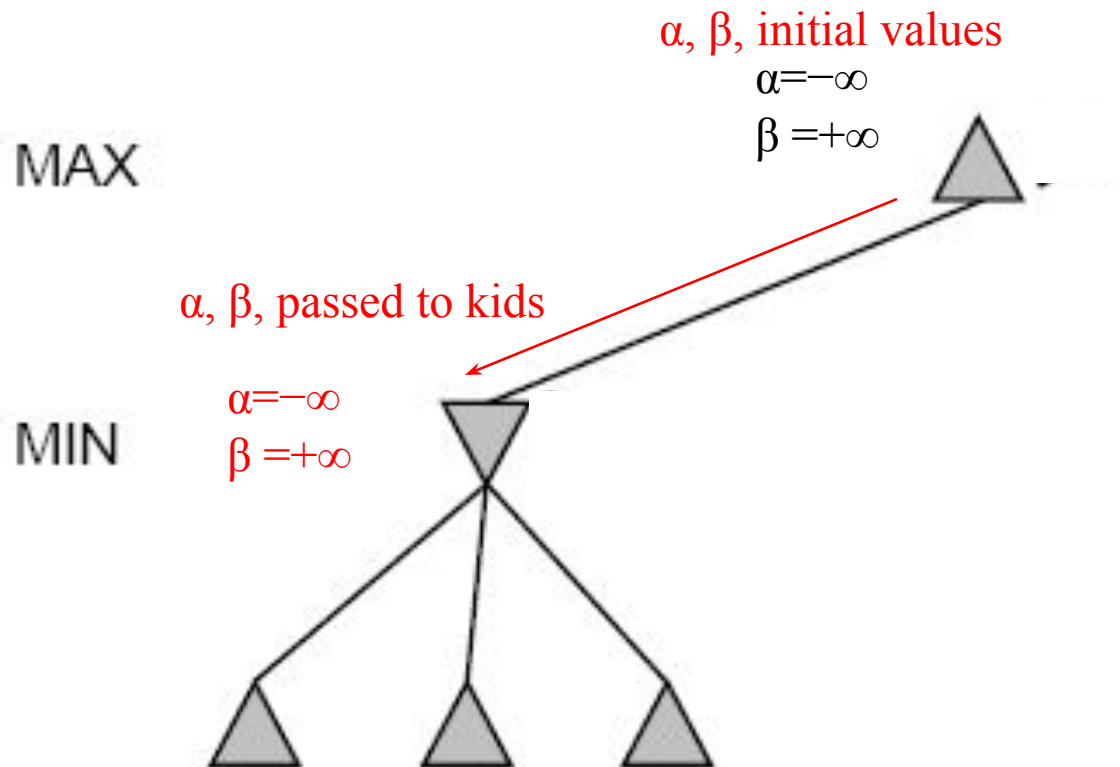
Prune whenever $\alpha \geq \beta$.

- Prune below a Max node whose alpha value becomes greater than or equal to the beta value of its ancestors.
 - **Max nodes update alpha** based on children's returned values.
- Prune below a Min node whose beta value becomes less than or equal to the alpha value of its ancestors.
 - **Min nodes update beta** based on children's returned values.

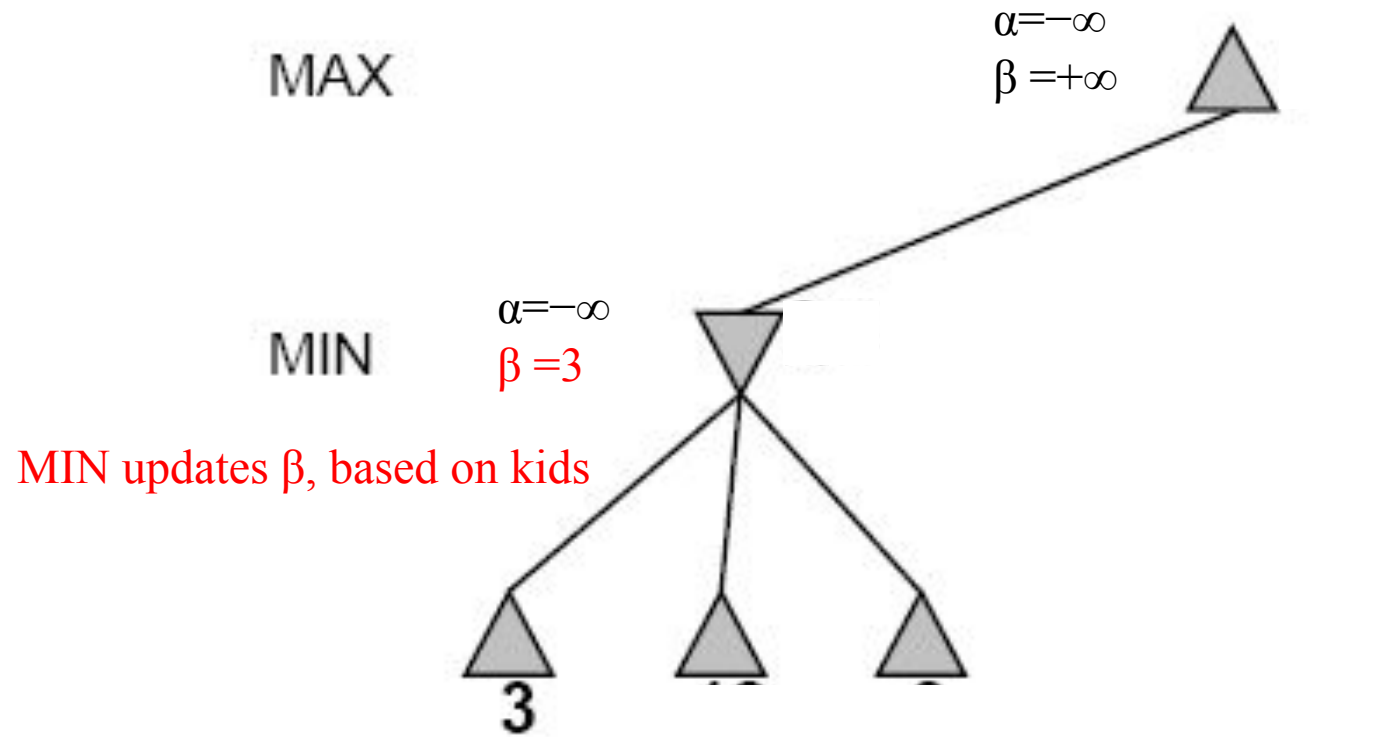


Alpha-Beta Example Revisited

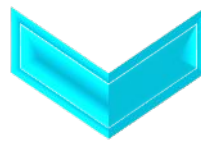
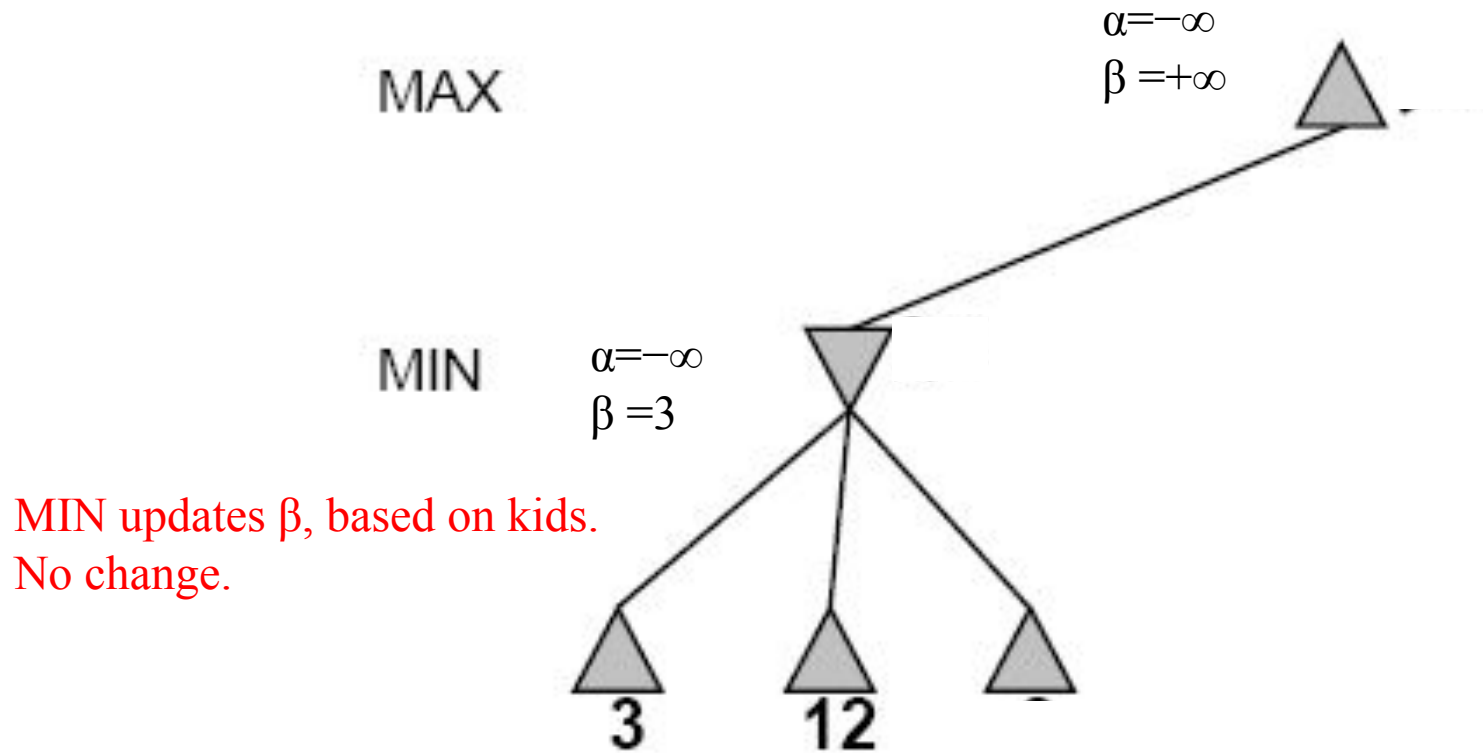
Do DF-search until first leaf



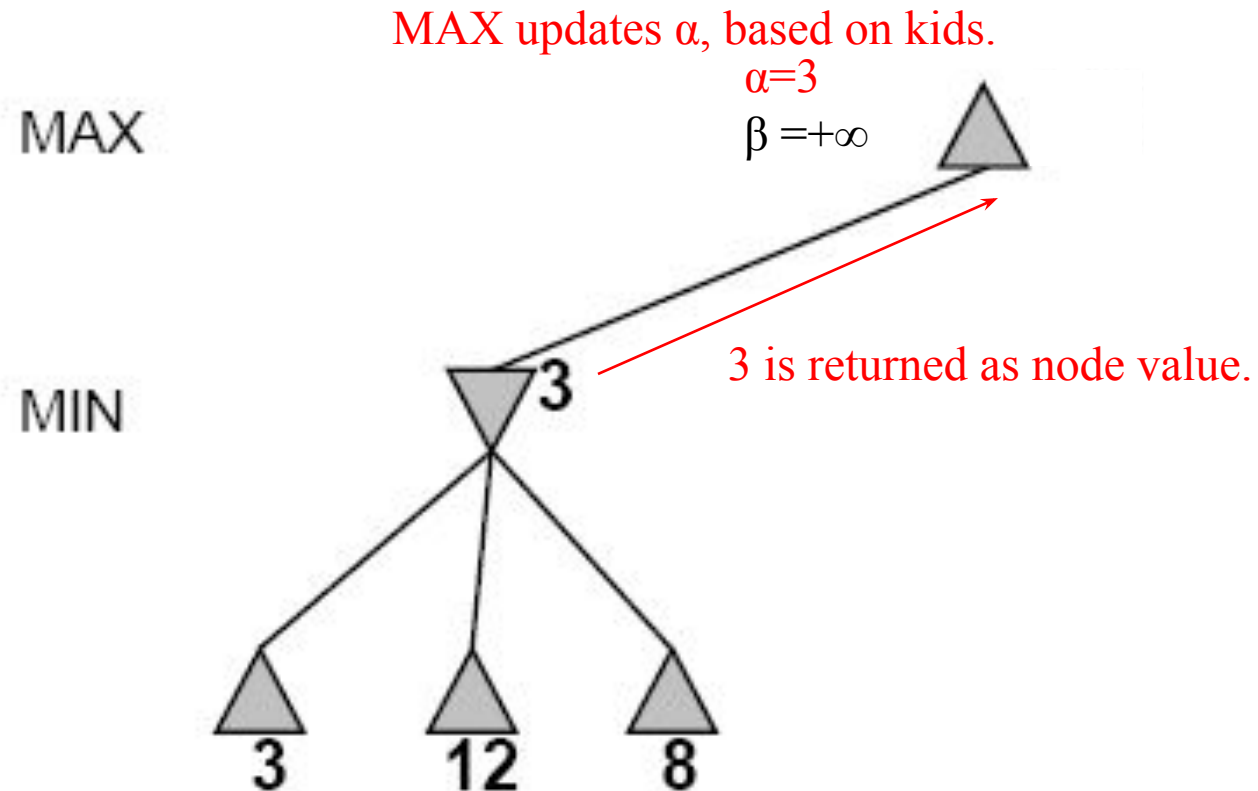
Alpha-Beta Example Revisited



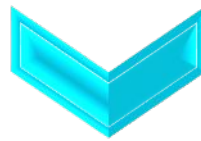
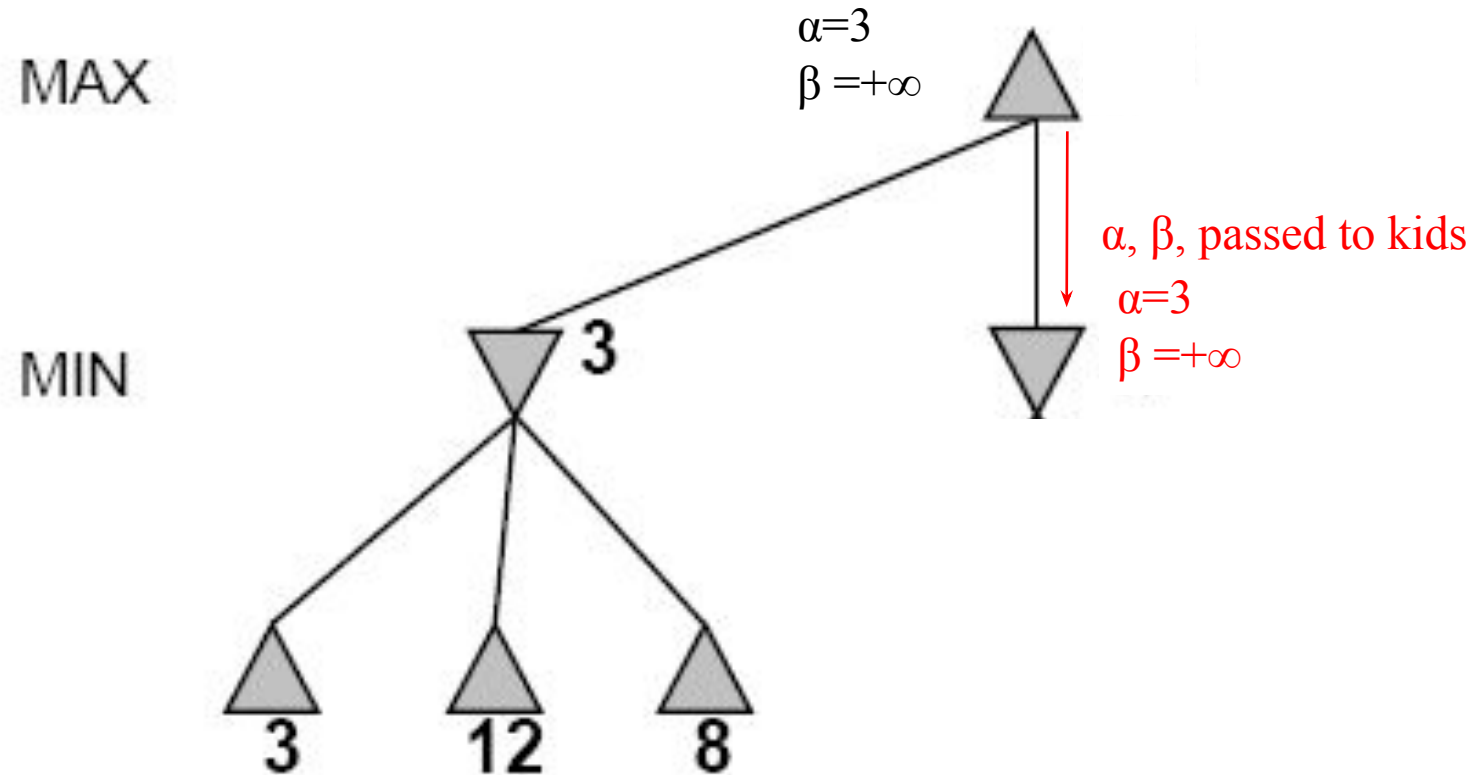
Alpha-Beta Example Revisited



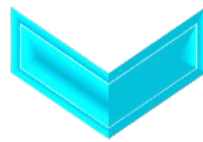
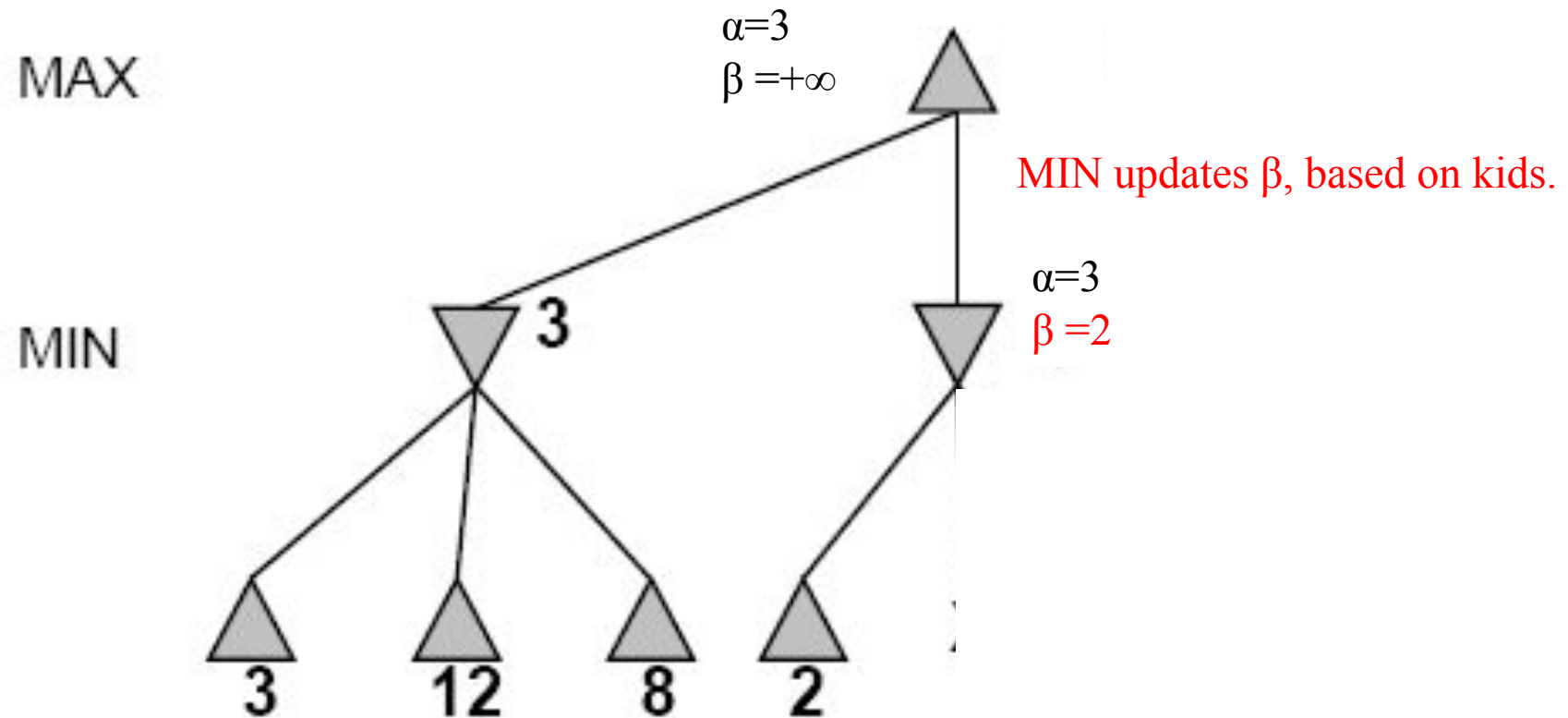
Alpha-Beta Example Revisited



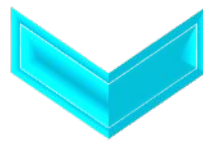
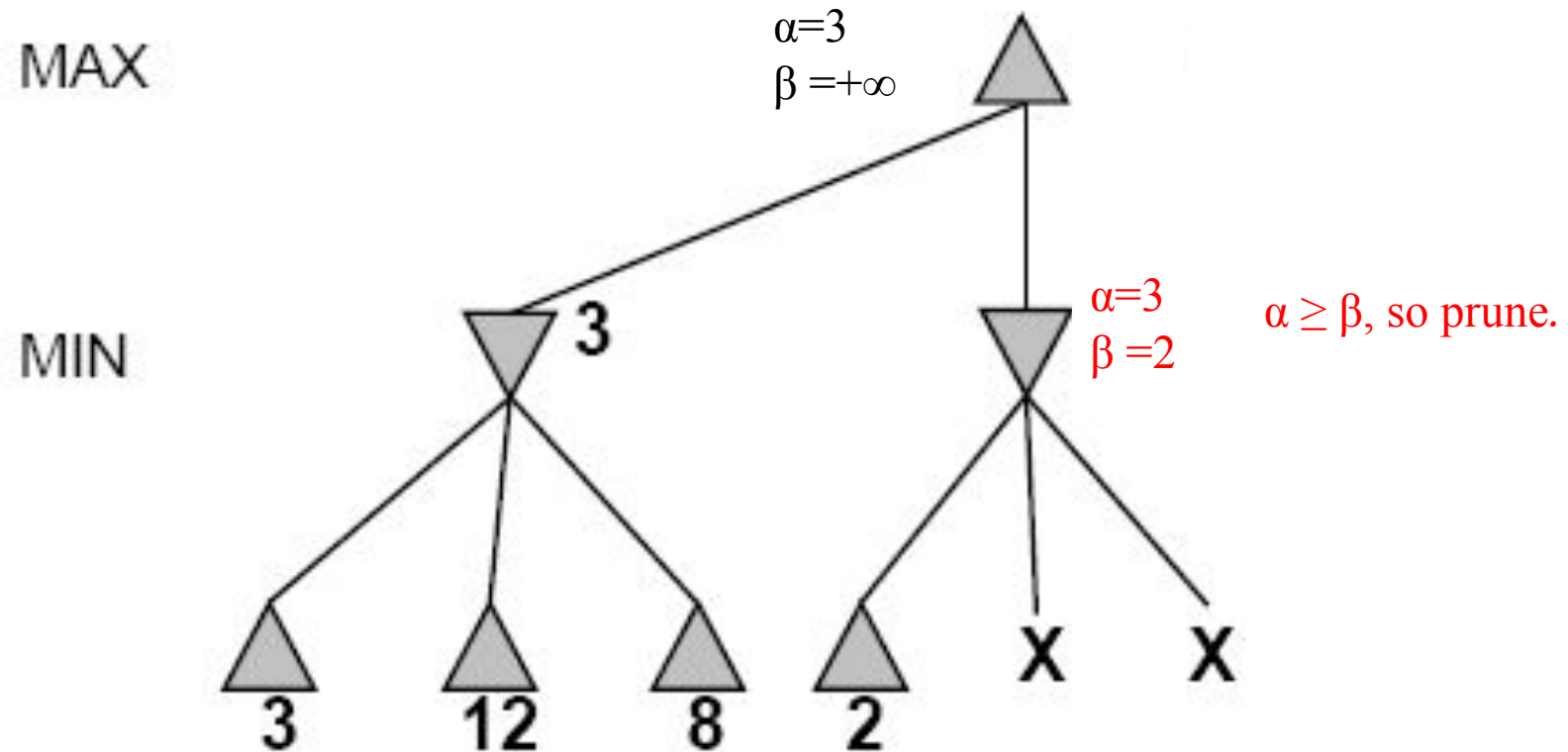
Alpha-Beta Example Revisited



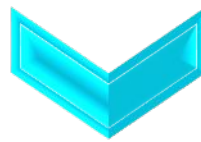
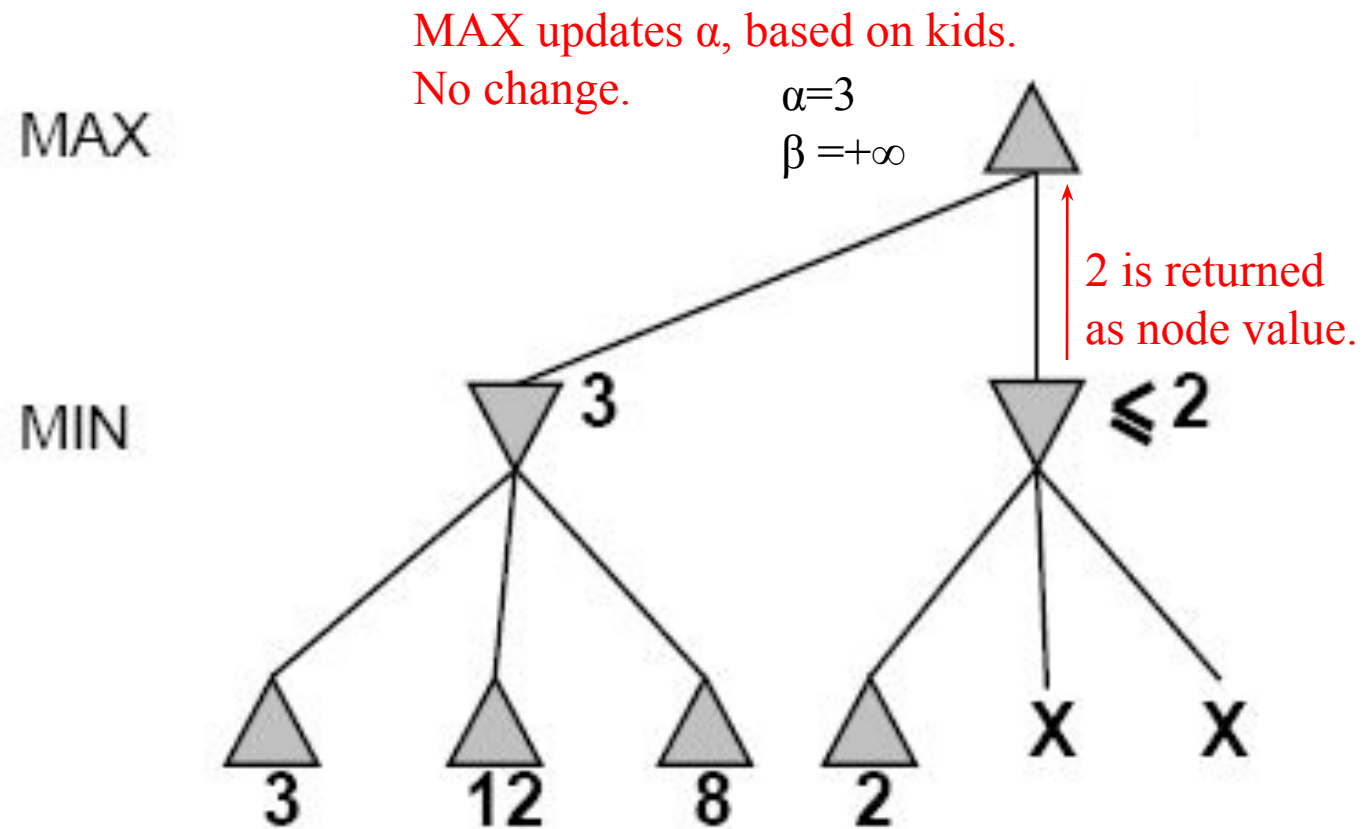
Alpha-Beta Example Revisited



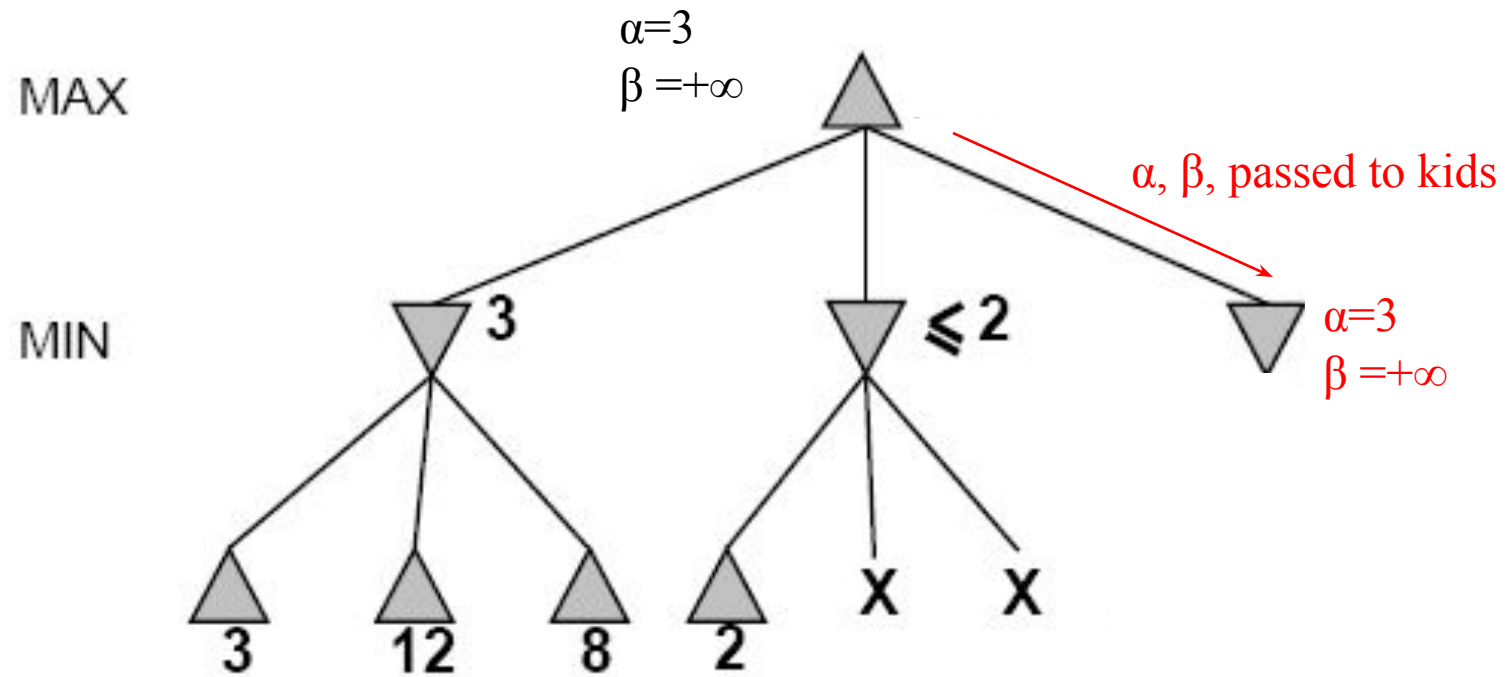
Alpha-Beta Example Revisited



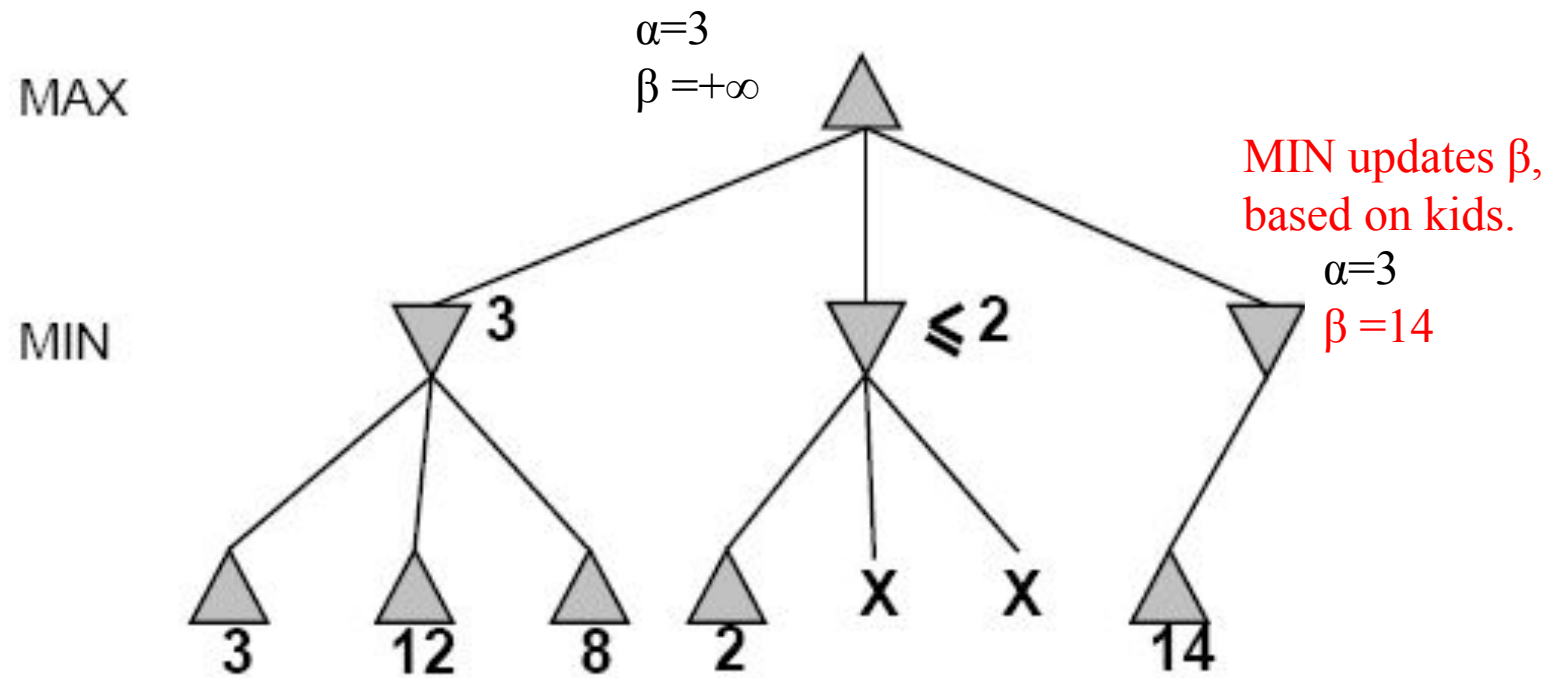
Alpha-Beta Example (continued)



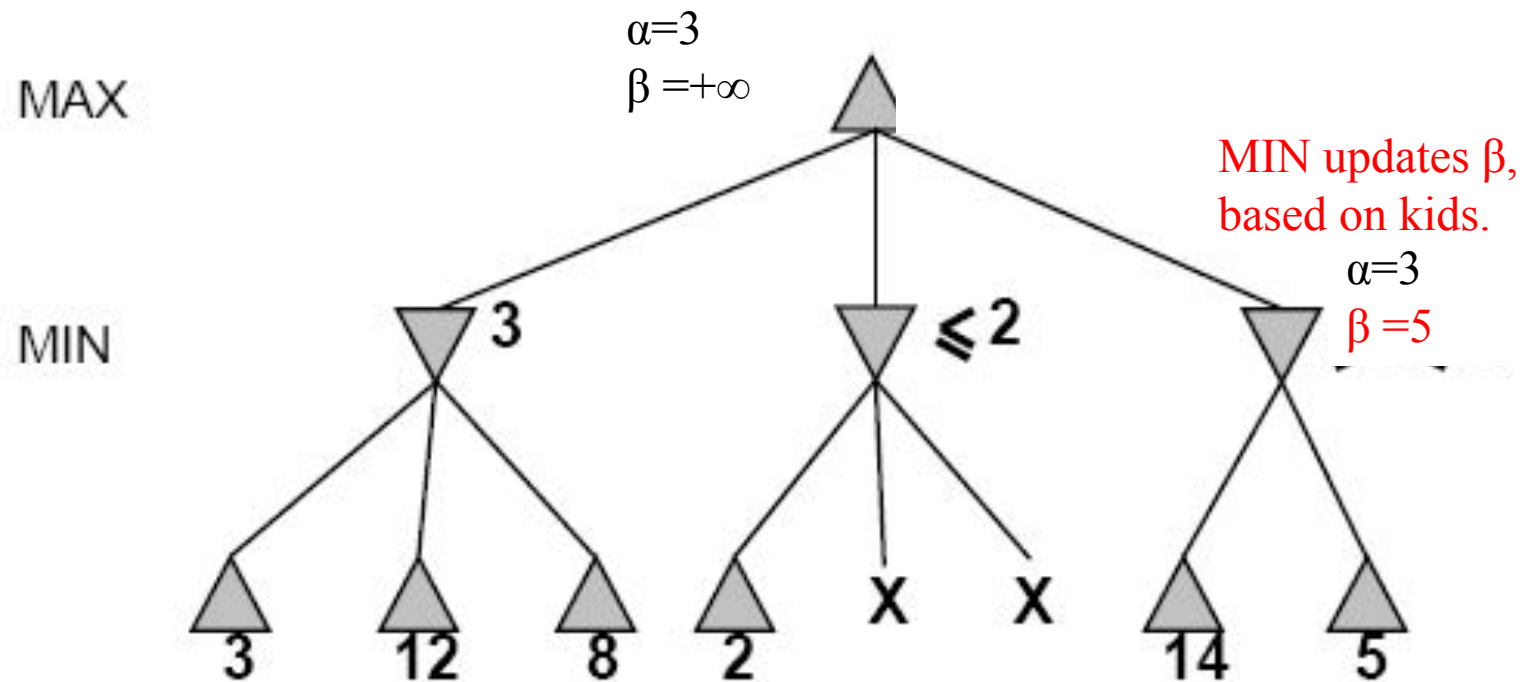
Alpha-Beta Example Revisited



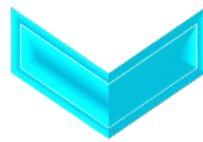
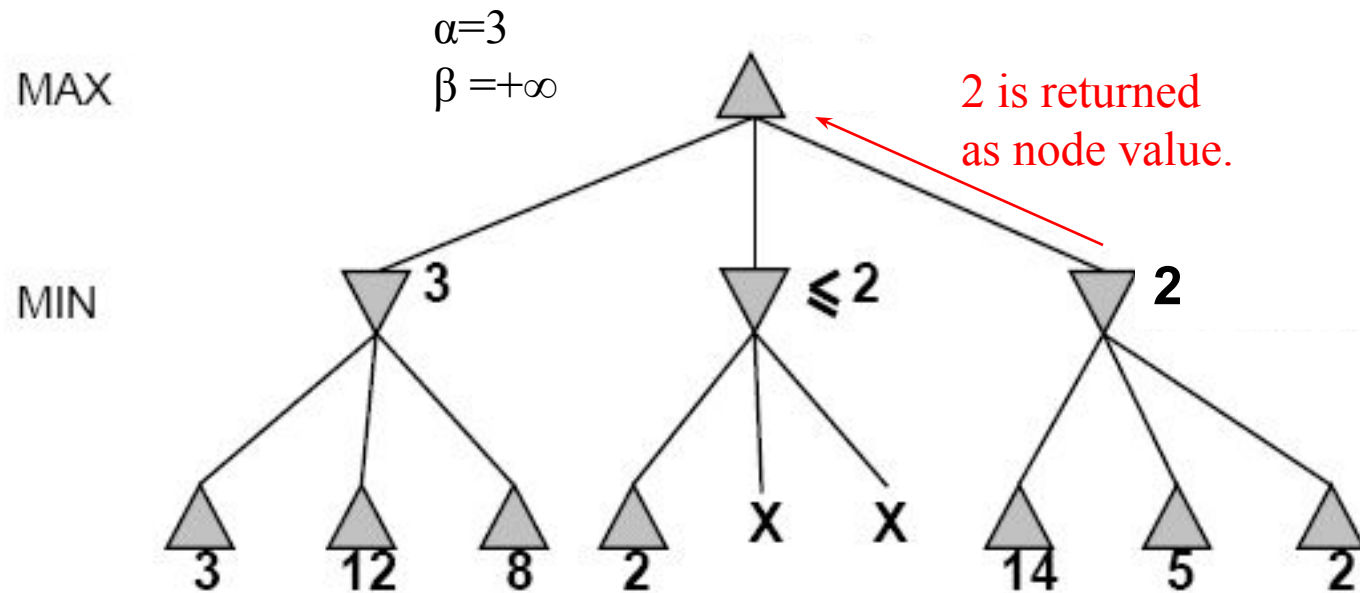
Alpha-Beta Example Revisited



Alpha-Beta Example Revisited

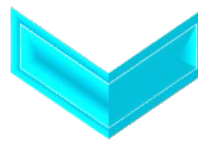
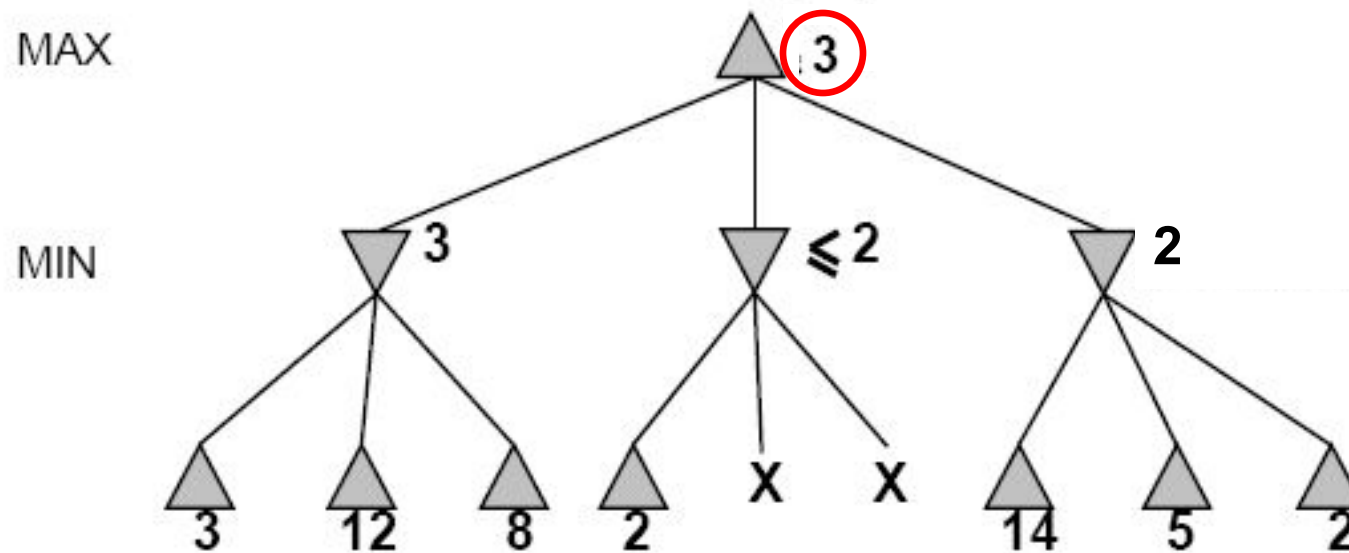


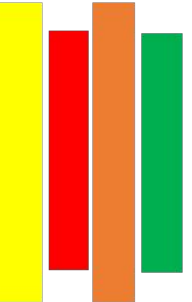
Alpha-Beta Example Revisited



Alpha-Beta Example Revisited

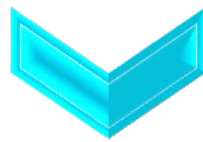
Max calculates the same node value, and makes the same move!

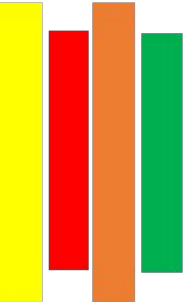




Analysis

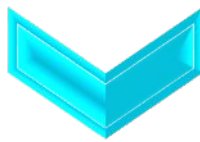
- Worst-Case
 - branches are ordered so that no pruning takes place. In this case alpha-beta gives no improvement over exhaustive search
- Best-Case
 - each player's best move is the left-most child (i.e., evaluated first)
 - in practice, performance is closer to best rather than worst-case
 - E.g., sort moves by the remembered move values found last time.
 - E.g., expand captures first, then threats, then forward moves, etc.
 - E.g., run Iterative Deepening search, sort by value last iteration.
- In practice often get $O(b^{(d/2)})$ rather than $O(b^d)$



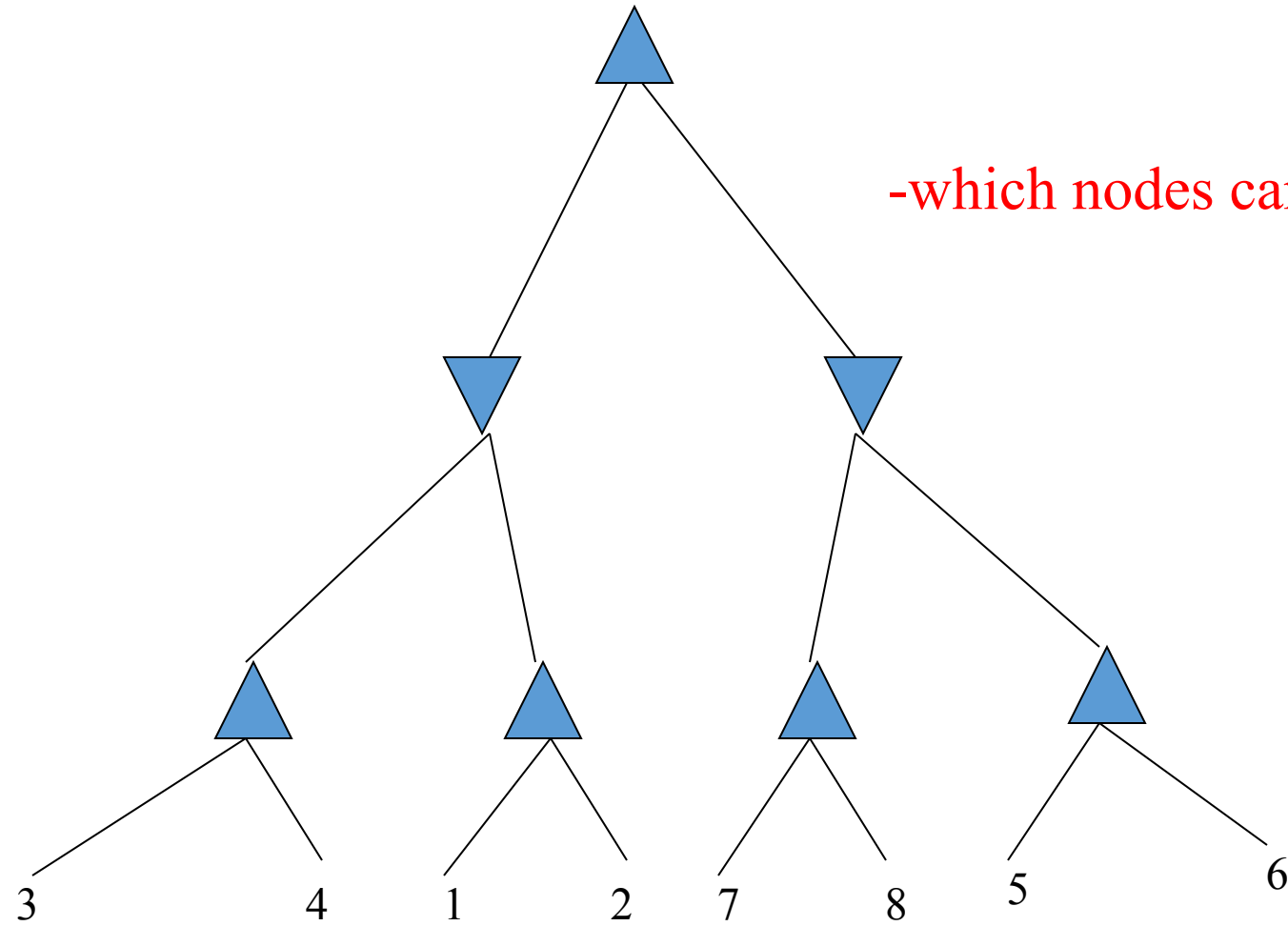


Final Comments about Alpha-Beta Pruning

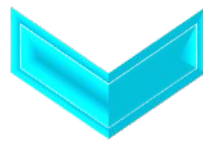
- Pruning does not affect final results
- Entire subtrees can be pruned.
- Good move *ordering* improves effectiveness of pruning
- Repeated states are again possible.
 - Store them in memory = transposition table



Practice Problem



-which nodes can be pruned?





thank
you